



AEGIS Architecture

Anonymized Exchange Gateway for Imaging Studies

SENDING SITE (Hospital Browser — Zero Install)

React Upload Portal (PWA)

- DICOM file/folder picker
- Client-side parsing (dcmjs)
- Tag de-identification (PS3.15)
- Anonymization preview
- Chunked upload to GCS

De-identification Engine

- HIPAA Safe Harbor (18 identifiers)
- D/Z/X/U/C action codes per tag
- Deterministic UID hashing
- Date shifting
- Private tag removal

Uploader Library (TypeScript)

- Reusable npm package
- Web Workers for parsing
- Embeddable in other apps
- Progress tracking

Key Principle

PHI is stripped in the browser
BEFORE data leaves the
hospital network.

Only tag-de-identified DICOM
is transmitted over HTTPS/TLS.

No software installation required.

HTTPS (TLS 1.2+) — De-identified DICOM only

GCP PROJECT (Secured Enterprise Tenancy)

Cloud Armor (DDoS / WAF)

+ Global HTTPS Load Balancer

+ Identity-Aware Proxy (admin routes)

API Backend (Go / Cloud Run)

- Upload orchestration (signed URLs)
- Study / project management
- User auth (OAuth 2.0 / JWT)
- Routing rules engine
- DICOMweb proxy ☐ Healthcare API
- Pub/Sub event handlers
- distroless image (~10-20 MB)

GCP Healthcare API

- DICOM Store (DICOMweb)
- STOW-RS / WADO-RS / QIDO-RS
- Server-side de-id validation
- STOW/WADO type detection (DLP)
- Pub/Sub notifications
- BigQuery metadata export
- CMEK encryption at rest

Cloud Storage

- Staging bucket (uploads)
- Archive bucket (exports)
- Signed URL uploads
- Lifecycle policies

Pub/Sub

- Ingest notifications
- Defacing triggers
- Routing events
- Audit events

HTTP trigger

Defacing Service (Python / Cloud Run)

- Triggered for head MRI/PET/CT
- WADO-RS retrieval from DICOM store
- dcm2nix (DICOM ☐ NIfTI)
- mri_deface or DeepDefacer
- pydicom pixel data injection
- STOW-RS back to 'clean' store
- ~0.5-2 GB container image
- 2-10 min per volume

Admin Dashboard (React / Cloud Run)

- Behind Identity-Aware Proxy (IAP)
- OHIF Viewer (DICOMweb)
- Study browser + QC review
- Defacing review (before/after)
- Routing rule configuration
- User/institution management
- Audit log viewer

Two DICOM Stores

raw Tag de-id'd, not defaced

clean Fully processed + defaced

Non-destructive pipeline:
raw ☐ deface ☐ clean ☐ approve

Admin reviews before release

Security Layers

- VPC Service Controls
- Cloud Armor DDoS/WAF
- TLS 1.2+ everywhere
- CMEK (Cloud KMS)
- IAM least privilege
- Cloud Audit Logs
- Secret Manager
- Artifact Registry scanning
- distroless containers

Repository Structure (5 Repos)

- | | |
|--|--|
|  aegis-terraform-prj | GCP project bootstrap, IAM, KMS |
|  aegis-terraform-infra | VPC, Cloud Run, Healthcare API, Cloud Armor |
|  aegis-api | Go backend — upload, routing, DICOMweb proxy |
|  aegis-frontend | React — Upload Portal + Admin Dashboard |
|  aegis-client | TypeScript uploader library (npm package) |

Key Open-Source Dependencies

- | | |
|------------------|--|
| Go: | suyashkumar/dicom, GCP Go SDK |
| Browser: | dcmjs, dicomParser, OHIF Viewer |
| Defacing: | mri_deface, dcm2nix, pydicom, nibabel |
| Infra: | Terraform Google Provider, Cloud Build |
| Viewing: | OHIF Viewer (MIT) — DICOMweb native |

Implementation Phases

Phase 1: Foundation MVP

Terraform + Go API + Upload Portal + Admin Dashboard

Phase 2: Defacing Pipeline

Python Cloud Run sidecar, mri_deface, automated trigger

Phase 3: Operations

Routing rules, institution management, audit logging

Phase 4: Advanced

Burned-in PHI OCR, QC automation, BIDS conversion